

ODOO DEVELOPER PROGRAM

Environment & Decorators

Odoo 17 Development Tutorials

Ahmed Muhumed

July 25, 2026

OUTLINE

- 1 Introduction to the Environment
- 2 `self.env` Explained
- 3 Environment Components
- 4 Accessing Models with `env[...]`
- 5 User and Company
- 6 Context (`env.context`)
- 7 `with_*` Environment Methods
- 8 `sudo()` Superuser Access
- 9 `env.ref()` and XML IDs
- 10 Introduction to API Decorators
- 11 `@api.model` and
`@api.model_create_multi`
- 12 `@api.depends`
- 13 `@api.onchange`
- 14 `@api.constrains`
- 15 Other Important Decorators
- 16 Summary

Introduction to the Environment

WHAT WILL WE LEARN?

- ▶ What **`self.env`** is and why every Odoo method uses it.
- ▶ The main parts of the environment: user, company, context, cursor, and registry.
- ▶ How to switch environment safely with `with_context`, `with_user`, `sudo()`, and friends.
- ▶ How to resolve XML IDs with `env.ref()`.
- ▶ The most important **`@api`** decorators and when to use each one.

WHY THE ENVIRONMENT MATTERS

- ▶ In Odoo, business code almost never talks to PostgreSQL directly.
- ▶ Every recordset carries an **environment**.
- ▶ The environment answers questions such as:
 - Who is the current user?
 - Which company is active?
 - What language / context flags are active?
 - Which database cursor should be used?
- ▶ Understanding `self.env` is the foundation for correct ORM code.

self.env Explained

WHAT IS `self.env`?

- ▶ `self.env` is the **Environment** object attached to a recordset.
- ▶ It is the gateway to models, users, companies, context, and the database cursor.

Basic Access

```
1 class Student(models.Model):  
2     _name = 'student.student'  
3  
4     def action_demo(self):  
5         print(self.env)  
6         students = self.env['student.student'].search([])
```

- ▶ Inside model methods, you almost always start from `self.env`.

ENVIRONMENT VS RECORDSET

- ▶ A **recordset** represents one or more records of a model.
- ▶ An **environment** is the execution context around those records.

```
1 student = self.env['student.student'].browse(15)
2 print(student)      # student.student(15,)
3 print(student.env)  # Environment object
```

- ▶ Different recordsets may share the same environment.
- ▶ Changing the environment (for example with `sudo()`) returns a **new** recordset / env pair.

WHERE DOES `SELF.ENV` COME FROM?

- ▶ Odoo builds the environment when a request / shell / cron job starts.
- ▶ Then every recordset created in that flow receives that environment.

```
1 # In Odoo shell
2 env[ 'student.student' ].search ([])
3
4 # In a model method
5 self.env[ 'student.student' ].search ([])
```

- ▶ In both cases, `env` is the same kind of object.
- ▶ In model methods, it is available as **`self.env`**.

Environment Components

MAIN PARTS OF THE ENVIRONMENT

► The environment contains several important pieces:

- 1 `env.cr` – database cursor
- 2 `env.uid` – current user ID
- 3 `env.user` – current user record
- 4 `env.context` – context dictionary
- 5 `env.company` / `env.companies`
- 6 `env.registry` – model registry
- 7 `env.su` – whether sudo mode is active

- ▶ `env.cr` is the PostgreSQL cursor for the current transaction.

```
1 self.env.cr.execute("SELECT COUNT(*) FROM student_student")  
2 count = self.env.cr.fetchone()[0]
```

- ▶ Prefer ORM methods over raw SQL whenever possible.
- ▶ Use `env.cr` only for advanced queries/reports.
- ▶ Odoo manages commit/rollback for HTTP requests automatically.

BREAKING DOWN `ENV.UID` AND `ENV.SU`

- ▶ **`env.uid`**: integer ID of the current user.
- ▶ **`env.su`**: `True` if the environment is in superuser mode.

```
1 print(self.env.uid)    # e.g. 2  
2 print(self.env.su)    # False in normal requests
```

- ▶ After `sudo()`, `env.su` becomes `True`.
- ▶ Access rights and record rules depend on these values.

BREAKING DOWN `ENV.REGISTRY`

- ▶ `env.registry` holds all loaded models for the database.

```
1 Student = self.env.registry[ 'student.student' ]
```

- ▶ In normal business code, prefer:

```
1 Student = self.env[ 'student.student' ]
```

- ▶ The registry is rebuilt when modules are installed or updated.

Accessing Models with env[. . .]

THE PATTERN `SELF.ENV[ 'MODEL.NAME' ]`

- ▶ This is the standard way to get a model from the environment.

Method Syntax

```
1 Model = self.env[ 'student.student' ]  
2 records = Model.search ( [])
```

- ▶ `self.env[ 'student.student' ]` returns an empty recordset of that model in the current environment.
- ▶ From there you call ORM methods: `create`, `search`, `browse`, ...

PRACTICAL EXAMPLES

```
1 # Search students
2 students = self.env['student.student'].search([( 'active ', '=', True) ])
3
4 # Create a classroom
5 classroom = self.env['school.classroom'].create({
6     'name': 'Grade 12-A',
7 })
8
9 # Browse by ID
10 partner = self.env['res.partner'].browse(7)
```

- Always use the technical model name (`_name`), not the Python class name.

SAME MODEL, DIFFERENT ENVIRONMENTS

```
1 students = self.env[ 'student.student' ]  
2 admin_students = self.env[ 'student.student' ].sudo()  
3 company_students = self.env[ 'student.student' ].with_company(company)
```

- ▶ All three access the same model.
- ▶ But rights, company, or context may differ.
- ▶ This is why environment control is so important.

User and Company

- **env.user** is the current `res.users` record.

```
1 user = self.env.user
2 print(user.name)
3 print(user.login)
4 print(user.has_group('base.group_system'))
```

- Useful for:
- checking groups / permissions in Python
 - writing the current user into a field
 - personalizing business logic

- ▶ **env.company**: the active company.
- ▶ **env.companies**: allowed companies in the current environment.

```
1 company = self.env.company
2 print(company.name)
3 print(self.env.companies.mapped('name'))
```

- ▶ Multi-company rules often depend on these values.
- ▶ Never hard-code company IDs when `env.company` is enough.

PRACTICAL USER / COMPANY PATTERNS

```
1 def create(self, vals):  
2     vals.setdefault('company_id', self.env.company.id)  
3     vals.setdefault('user_id', self.env.user.id)  
4     return super().create(vals)
```

- ▶ Set defaults from the environment.
- ▶ Keep business documents linked to the correct company/user.

Context (`env.context`)

WHAT IS THE CONTEXT?

- ▶ **env.context** is a dictionary of flags and values for the current operation.

```
1 print(self.env.context)
2 # example: { 'lang': 'en-US', 'tz': 'Africa/Mogadishu', 'uid': 2 }
```

- ▶ Context can change:
 - language
 - default values
 - active test behavior
 - custom business flags

READING CONTEXT VALUES

```
1 lang = self.env.context.get('lang')  
2 force_email = self.env.context.get('force_send_email')  
3 default_gender = self.env.context.get('default_gender')
```

- ▶ Use `.get()` so missing keys do not crash your code.
- ▶ Keys starting with `default_` are used by `default_get()`.

Method Syntax

```
1 records.with_context(key=value, ...)  
2 # or  
3 records.with_context({...})
```

```
1 students = self.env['student.student'].with_context(  
2     default_gender='female',  
3     active_test=False,  
4 )
```

- ▶ Returns a new recordset/environment with updated context.
- ▶ Original environment is not modified.

COMMON CONTEXT KEYS

- ▶ `lang` – language code
- ▶ `tz` – timezone
- ▶ `active_test` – whether to hide inactive records
- ▶ `default_fieldname` – default value for create forms
- ▶ custom keys for your own business logic

```
1 self.with_context(skip_student_validation=True).write(vals)
```

with_* Environment Methods

- ▶ Runs the next operations as another user.

```
1 other_user = self.env[ 'res.users' ].browse(8)
2 students = self.env[ 'student.student' ].with_user( other_user ).search ( [])
```

- ▶ Useful for testing access rights or impersonation flows.
- ▶ Access rights and record rules follow the selected user.

WITH_COMPANY () AND WITH_COMPANIES ()

```
1 company = self.env[ 'res.company' ].browse(2)
2 Student = self.env[ 'student.student' ].with_company(company)
3 Student.create({ 'name': 'Ahmed', 'company_id': company.id })
```

- ▶ Changes the active company in the environment.
- ▶ Important in multi-company databases.
- ▶ Prefer this over manually guessing company context.

WITH_ENV () AND CHAINING

```
1 new_env = self.env(user=other_user, context=dict(self.env.context, lang='so_SO'))
2 students = self.env['student.student'].with_env(new_env).search([])
```

- ▶ `with_env()` replaces the whole environment.
- ▶ You can also chain helpers:

```
1 self.env['student.student'] \
2     .with_user(user) \
3     .with_company(company) \
4     .with_context(lang='en-US') \
5     .search([])
```

IMPORTANT RULE

- ▶ Environment helpers return **new** recordsets.
- ▶ They do not mutate the old one in place.

```
1 a = self.env[ 'student.student ']  
2 b = a.sudo()  
3 print(a.env.su)   # False  
4 print(b.env.su)   # True
```

- ▶ Always keep using the returned value.

sudo () Superuser Access

WHAT IS `SUDO()` ?

- ▶ `sudo()` returns the same records in a superuser environment.
- ▶ Superuser bypasses access rights and record rules.

Method Syntax

```
1 records.sudo()  
2 records.sudo(flag=True)  
3 records.sudo(False)  # leave sudo mode
```

WHEN TO USE `SUDO()`

- ▶ System operations that normal users must trigger but cannot fully access.
- ▶ Creating technical records (followers, logs, queues) in controlled code.
- ▶ Cron / server actions that need broader access.

```
1 # Controlled technical write  
2 self.sudo().write({ 'system_note': 'Processed by cron' })
```

WHEN NOT TO USE `SUDO()`

- ▶ Do not use `sudo()` just to “make the error go away”.
- ▶ Do not expose `sudo` results directly to untrusted UI input.
- ▶ Prefer correct access rights and record rules when possible.

```
1 # Dangerous pattern  
2 self.env[ 'res.partner' ].sudo().search([]).unlink()
```

- ▶ Ask: does this really need superuser power?

SUDO () VS WITH_USER ()

- ▶ `sudo()`: bypass security (superuser).
- ▶ `with_user(user)`: act as that user, still under that user's rights.

```
1 # Bypass rights
2 self.sudo().write({'state': 'done'})
3
4 # Act as teacher user, respecting rights
5 self.with_user(teacher).write({'state': 'done'})
```

`env.ref()` and XML IDs

WHAT IS `ENV.REF()` ?

- ▶ `env.ref()` finds a record by its XML / external ID.

Method Syntax

```
1 record = self.env.ref('module_name.xml_id')  
2 record = self.env.ref('module_name.xml_id', raise_if_not_found=False)
```

- ▶ Extremely common for groups, stages, sequences, default records, and actions.

PRACTICAL EXAMPLES

```
1 admin_group = self.env.ref( 'base.group-system' )
2 student_action = self.env.ref( 'school-manager.action-student' )
3 draft_stage = self.env.ref( 'project.project-stage-0', raise_if_not_found=False)
4
5 if self.env.user.has_group( 'base.group-system' ):
6     # admin-only logic
7     pass
```


WHY PREFER XML IDS OVER HARD-CODED IDS?

- ▶ Database IDs change between databases.
- ▶ XML IDs are stable across installs when defined in data files.

```
1 # Bad
2 stage = self.env[ 'project.task.type' ].browse(3)
3
4 # Good
5 stage = self.env.ref( 'project.project_stage_0' )
```

- ▶ Use `raise_if_not_found=False` when the record may be optional.

OTHER USEFUL ENVIRONMENT HELPERS

```
1 self.env.lang           # current language, e.g. 'en_US'  
2 self.env.is_superuser()  
3 self.env.is_admin()  
4 self.env.is_system()
```

- ▶ These helpers make environment checks clearer and safer.

Introduction to API Decorators

WHAT ARE API DECORATORS?

- ▶ In Odoo, methods are often decorated with `@api . . . .` markers.
- ▶ A decorator tells Odoo **how** the method should be called and when it should run.
- ▶ Decorators are not just Python sugar – they change ORM behavior.

COMMON DECORATORS WE WILL COVER

- ▶ `@api.model`
- ▶ `@api.model_create_multi`
- ▶ `@api.depends`
- ▶ `@api.onchange`
- ▶ `@api.constrains`
- ▶ `@api.ondelete`
- ▶ `@api.autovacuum`
- ▶ `@api.private` / `@api.readonly`

WHY DECORATORS MATTER

- ▶ They decide whether a method is model-level or record-level.
- ▶ They connect computed fields to their dependencies.
- ▶ They validate data before it is saved.
- ▶ They control UI onchange behavior in forms.
- ▶ Using the wrong decorator creates subtle bugs.

`@api.model` and

`@api.model_create_multi`

- ▶ Marks a method as a **model-level** method.
- ▶ `self` is an empty recordset of the model, not one specific record.

Method Syntax

```
1 @api.model
2 def my_method(self, values):
3     return self.create(values)
```

- ▶ Common for helpers that create/search without needing an existing record.

BREAKING DOWN @API.MODEL

```
1 @api.model
2 def create_demo_student(self):
3     print(self) # student.student()
4     return self.create({'name': 'Demo Student'})
```

- Call it from the model:

```
1 self.env['student.student'].create_demo_student()
```

- Do not expect `self.id` to exist inside a pure model method.

- ▶ Used for efficient multi-record creation.
- ▶ The argument is a **list of dictionaries**.

```
1 @api.model_create_multi
2 def create(self, vals_list):
3     for vals in vals_list:
4         vals.setdefault('active', True)
5     return super().create(vals_list)
```

- ▶ Preferred modern override for `create()` in Odoo 17.

@API.MODEL VS RECORD METHODS

► Model method:

```
1 @api.model
2 def get_active_count(self):
3     return self.search_count([( 'active ', '=', True)])
```

► Record method:

```
1 def action_archive(self):
2     self.ensure_one()
3     self.active = False
```

► Choose based on whether you need existing records.

`@api.depends`

WHAT IS @API.DEPENDS?

- ▶ Used on **computed fields**.
- ▶ Tells Odoo which fields trigger recomputation.

Method Syntax

```
1 age_group = fields.Char(compute='_compute_age_group')
2
3 @api.depends('age')
4 def _compute_age_group(self):
5     for student in self:
6         student.age_group = 'adult' if student.age >= 18 else 'minor'
```

BREAKING DOWN DEPENDENCIES

```
1 @api.depends( 'birth_date' )
2 def _compute_age( self ):
3     ...
4
5 @api.depends( 'line_ids.price', 'line_ids.quantity' )
6 def _compute_total( self ):
7     ...
8
9 @api.depends( 'classroom_id.teacher_id.name' )
10 def _compute_teacher_name( self ):
11     ...
```

- ▶ You can depend on related paths and one2many subfields.
- ▶ Missing dependencies cause stale computed values.

STORED VS NON-STORED COMPUTED FIELDS

```
1 # Recomputed on the fly  
2 display_label = fields.Char(compute='_compute_display_label')  
3  
4 # Stored in database and recomputed when deps change  
5 total = fields.Float(compute='_compute_total', store=True)
```

- ▶ `store=True` **needs accurate** `@api.depends`.
- ▶ Non-stored fields are calculated when read.

INVERSE AND RELATED NOTES

```
1 @api.depends('first_name', 'last_name')
2 def _compute_name(self):
3     for rec in self:
4         rec.name = f"{rec.first_name} {rec.last_name}"
5
6 def _inverse_name(self):
7     for rec in self:
8         parts = (rec.name or '').split(' ', 1)
9         rec.first_name = parts[0]
10        rec.last_name = parts[1] if len(parts) > 1 else ''
```

- Inverse methods allow writing back into computed fields.

@api.onchange

WHAT IS @API.ONCHANGE?

- ▶ Runs in the **UI form** when a field value changes.
- ▶ Used to update other fields before the record is saved.

Method Syntax

```
1 @api.onchange( 'classroom_id' )
2 def _onchange_classroom_id( self ):
3     if self.classroom_id:
4         self.teacher_id = self.classroom_id.teacher_id
```

BREAKING DOWN ONCHANGE BEHAVIOR

- ▶ Onchange methods work on a temporary record in the form client.
- ▶ They do **not** automatically run during pure Python `create()`/`write()` calls.
- ▶ Return warnings or domain updates when needed:

```
1 @api.onchange( 'age' )
2 def _onchange_age( self ):
3     if self.age and self.age < 0:
4         return {
5             'warning': {
6                 'title': 'Invalid Age',
7                 'message': 'Age cannot be negative. ',
8             }
9         }
```

RETURNING DOMAIN FROM ONCHANGE

```
1 @api.onchange( 'school_id' )
2 def _onchange_school_id( self ):
3     self.classroom_id = False
4     return {
5         'domain': {
6             'classroom_id': [( 'school_id', '=', self.school_id.id )]
7         }
8     }
```

- Useful for dynamic dropdown filtering in forms.

@API.ONCHANGE VS @API.DEPENDS

- ▶ @api.onchange: UI helper before save.
- ▶ @api.depends: computation/recompute logic in ORM.

```
1 # UI convenience
2 @api.onchange( 'classroom_id' )
3 def _onchange_classroom_id( self ):
4     self.teacher_id = self.classroom_id.teacher_id
5
6 # Real stored/computed business value
7 @api.depends( 'classroom_id.teacher_id' )
8 def _compute_teacher_id( self ):
9     for rec in self:
10         rec.teacher_id = rec.classroom_id.teacher_id
```

`@api.constrains`

WHAT IS @API.CONSTRAINS?

- ▶ Used for **Python validation** when fields change.
- ▶ Runs during `create()` / `write()` when watched fields are involved.

Method Syntax

```
1 @api.constrains('age')
2 def _check_age(self):
3     for student in self:
4         if student.age < 0:
5             raise ValidationError("Age cannot be negative.")
```

BREAKING DOWN CONSTRAINT METHODS

```
1 from odoo.exceptions import ValidationError
2
3 @api.constrains('email')
4 def _check_email(self):
5     for student in self:
6         if student.email and '@' not in student.email:
7             raise ValidationError("Please provide a valid email.")
```

- ▶ Always loop over self.
- ▶ Raise `ValidationError` for business validation failures.
- ▶ Constraints must be deterministic and side-effect free.

MULTI-FIELD CONSTRAINTS

```
1 @api.constrains('start_date', 'end_date')
2 def _check_dates(self):
3     for rec in self:
4         if rec.start_date and rec.end_date and rec.end_date < rec.start_date:
5             raise ValidationError("End date must be after start date.")
```

- ▶ List every field that should trigger the check.
- ▶ If a field is missing from `@api.constrains`, changes to it may skip validation.

PYTHON CONSTRAINTS VS SQL CONSTRAINTS

- ▶ `@api.constrains`: flexible Python checks, rich error messages.
- ▶ `_sql_constraints`: database-level checks/uniqueness, very strong.

```
1 _sql_constraints = [  
2     ('email_unique', 'unique(email)', 'Email must be unique.'),  
3 ]
```

- ▶ Use both when needed.

Other Important Decorators

- Controls deletion restrictions in modern Odoo versions.

```
1 @api.ondelete(at_uninstall=False)
2 def _unlink_except_done(self):
3     for rec in self:
4         if rec.state == 'done':
5             raise UserError("Done records cannot be deleted.")
```

- Clearer than putting all logic only in `unlink()`.

- Marks a method for periodic cleanup by the autovacuum cron.

```

1 @api.autovacuum
2 def _gc_old_logs(self):
3     old = self.search([
4         ('create_date', '<', fields.Datetime.subtract(fields.Datetime.now(), days
5             =90)),
6         ])
7     old.unlink()

```

- Useful for temporary logs, transient leftovers, and cleanup tasks.

@API.PRIVATE AND @API.READONLY

```
1 @api.private
2 def _prepare_student_vals(self, values):
3     values = dict(values)
4     values.setdefault('active', True)
5     return values
6
7 @api.readonly
8 def get_report_data(self):
9     return self.read(['name', 'age'])
```

- ▶ `@api.private`: not available through RPC / external API.
- ▶ `@api.readonly`: declares a non-writing method (useful for routing/optimization).

```
1 @api.returns('self', lambda value: value.id)
2 def copy(self, default=None):
3     return super().copy(default)
```

- ▶ Documents / adapts return values for RPC compatibility.
- ▶ Seen on methods that return records but need IDs over XML-RPC.

Summary

ENVIRONMENT SUMMARY

- ▶ **`self.env`** is the gateway to models, users, companies, context, and the cursor.
- ▶ Use `env['model']`, `env.user`, `env.company`, `env.context`, and `env.ref()`.
- ▶ Switch environment with `with_context`, `with_user`, `with_company`, and `sudo()`.
- ▶ Environment helpers return new recordsets; keep the returned value.

DECORATOR SUMMARY

- ▶ `@api.model / @api.model_create_multi` – model-level methods and create overrides.
- ▶ `@api.depends` – computed field dependencies.
- ▶ `@api.onchange` – UI form updates before save.
- ▶ `@api.constrains` – Python validation on create/write.
- ▶ `@api.ondelete, @api.private, @api.readonly, @api.autovacuum` – specialized behaviors.

PRACTICAL CHECKLIST

- ▶ Need another model? → `self.env['model.name']`
- ▶ Need current user/company? → `env.user / env.company`
- ▶ Need temporary flags? → `with_context(...)`
- ▶ Need to bypass rights carefully? → `sudo()`
- ▶ Need stable record lookup? → `env.ref('module.xml_id')`
- ▶ Need compute / onchange / validation? → choose the correct decorator